Experiment 09: Automated Task Agent (Web Search & Summarization Workflow)

Name : Atharv Bhosale

Roll No : DS20

PRN : 2324000461

1. Aim

To design, implement, and test an automated agentic workflow that searches the web for information on a chosen topic and then summarizes the findings into a short brief, with no manual intervention between the two steps.

2. Tools, Platform & Framework Used

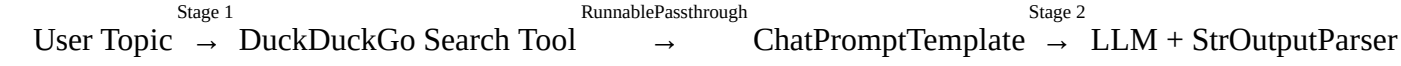
- **Operating System:** Linux (Ubuntu) with Python Virtual Environment (`venv`)
- **Coding Platform:** Jupyter Notebook
- **Language:** Python 3.12
- **Framework & Libraries:** `langchain`, `langchain-openai`, `langchain-community`, `ddgs` (`duckduckgo-search`)
- **Model & Endpoint:** OpenAI-compatible LLM via `ChatOpenAI` (Groq API Endpoint)

3. Theory & Pipeline Architecture

This experiment chains two capabilities together into a single automated pipeline:

1. **Stage 1 (Retrieval Step):** Gathers fresh, real-time information from the live web using DuckDuckGo search.
2. **Stage 2 (Generation Step):** Condenses the retrieved snippets into a structured executive brief using an LLM.

Unlike dynamic tool-calling agents where the LLM decides which tool to invoke in a loop, here we construct a **fixed two-stage sequential workflow** using LangChain Expression Language (LCEL):



Step 1: API Configuration & Dynamic Model Initialization

In this step, we securely load the API key using `getpass` and initialize `ChatOpenAI`. To prevent static model deprecation (`404 NotFoundError`) or quota exhaustion (`429`

`RateLimitError`), the script dynamically queries the OpenAI-compatible endpoint to select an active chat model.

```
import os
import getpass
from openai import OpenAI
from langchain_openai import ChatOpenAI

# 1. Enter your FREE Groq API Key (starts with gsk_...)
api_key = getpass.getpass("Enter your Groq API Key (starts with
gsk_...): ").strip()

# 2. Connect to Groq and automatically pick an active chat model
client = OpenAI(api_key=api_key,
base_url="https://api.groq.com/openai/v1")
available_models = [
    m.id for m in client.models.list().data
    if not any(x in m.id.lower() for x in ["whisper", "tts", "guard",
"embed", "vision"])
]
chosen_model = available_models[0]
print(f"Auto-selected available model: {chosen_model}")

# 3. Initialize LangChain's ChatOpenAI with the verified model
llm = ChatOpenAI(
    model=chosen_model,
    api_key=api_key,
    base_url="https://api.groq.com/openai/v1",
    temperature=0.3
)

# 4. Quick test to verify API connection
response = llm.invoke("Reply with the exact phrase: API is connected")
print(response.content)
```

```
Enter your Groq API Key (starts with gsk_...):  ·····
```

```
Auto-selected available model: allam-2-7b
API is connected.
```

متصلة وجاهزة (API) كخدمة ذكاء اصطناعي، يمكنني تأكيد أن الواجهة البرمجية للتطبيق للاستخدام. يرجى تزويدي بمزيد من التفاصيل حول المهمة المطلوبة أو السؤال الذي تود مساعدتي في الإجابة عنه.

Step 2: Stage 1 — Live Web Search Retrieval Tool

Here we define Stage 1 of the workflow. Because certain default search backends (such as Mojeek) can experience ISP/firewall timeouts, we implement a resilient wrapper

(`reliable_web_search`) around `DDGS` that automatically falls back across multiple search backends (`duckduckgo`, `lite`, `html`, `bing`, `brave`) and wraps it into a standard LangChain `Tool`.

```
from langchain_core.tools import Tool
from ddgs import DDGS

# Resilient search function that tries multiple backends in case one
is blocked by your ISP/firewall
def reliable_web_search(query: str) -> str:
    for backend in ["duckduckgo", "lite", "html", "bing", "brave"]:
        try:
            results = DDGS(timeout=15).text(query, max_results=4,
backend=backend)
            if results:
                return "\n\n".join(
                    [f"Title: {r.get('title', '')}\nSnippet:
{r.get('body', '')}" for r in results]
                )
        except Exception:
            continue
    raise RuntimeError("All search backends timed out. Check your
internet connection.")

# Wrap it as a standard LangChain Tool
search_tool = Tool(
    name="DuckDuckGo_Search",
    func=reliable_web_search,
    description="Searches the web for fresh information on a given
topic."
)

# Test Stage 1 with a sample topic
test_topic = "Latest developments in Artificial Intelligence agents"
print(f"Searching the web for: '{test_topic}'...\n")

raw_search_results = search_tool.invoke(test_topic)
print("--- Raw Search Results ---")
print(raw_search_results)
```

```
Searching the web for: 'Latest developments in Artificial Intelligence
agents'...

--- Raw Search Results ---
Title: Latest AI developments 2026: Models, agentic AI, infrastructure
Snippet: What are the latest AI developments this week? The latest AI
developments this week are Anthropic's agentic AI tool-use update and
OpenAI's model routing refinement, both shipped days after August's
earlier releases. This is artificial intelligence news worth a full
read, not a headline skim, since routing changes affect token cost
```

```
directly.

Title: Exploring the Future of AI Agents: Trends and Innovations in AI
Agent ...
Snippet: The artificial intelligence landscape is experiencing a
fundamental transformation as we progress through 2025. AI agents and
AI-ready data have emerged as the two fastest advancing technologies
on the 2025 Gartner Hype Cycle for Artificial Intelligence, signaling
a shift from generative AI as a standalone capability to autonomous
systems

Title: Latest Developments in Artificial Intelligence
Snippet: Discover the latest AI Innovations transforming industries
and enhancing technology in the United States. Stay informed with
cutting-edge developments today.

Title: Agentic AI Models: The Latest Developments and News
Snippet: Track the latest news, developments, and updates across 12
major agentic AI models including GPT, Claude, Gemini, DeepSeek,
Llama, Grok, and more.
```

Step 3: Stage 2 — Summarization Prompt & LLM Chain

Here we build Stage 2 of the workflow. We use `ChatPromptTemplate` to instruct the LLM to act as an expert research assistant and format the raw web snippets into a **2-sentence executive summary** followed by **3–4 key bullet points**, piped into `llm` and `StrOutputParser()`.

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. Create the prompt template for Stage 2 (Summarization)
summary_prompt = ChatPromptTemplate.from_template(
    """You are an expert research assistant.
Read the following web search results about '{topic}' and summarize
the key findings into a short, structured brief.

Include:
1. A 2-sentence executive summary.
2. 3 to 4 bullet points highlighting the most important facts or
trends.

Web Search Results:
{search_results}

Short Brief:"""
)
```

```python
# 2. Create the Stage 2 summarization chain
summarizer_chain = summary_prompt | llm | StrOutputParser()

# 3. Test Stage 2 using the search results from Cell 2
print("Generating summary brief...\n")
test_brief = summarizer_chain.invoke({
    "topic": test_topic,
    "search_results": raw_search_results
})

print("--- Stage 2 Output: Summarized Brief ---")
print(test_brief)
```

```
Generating summary brief...

--- Stage 2 Output: Summarized Brief ---
Executive Summary: The latest developments in Artificial Intelligence
(AI) agents highlight the growing focus on autonomous systems and
agentic AI models. In 2026, notable advancements include Anthropic's
agentic AI tool and OpenAI's model routing refinement, while the
Gartner Hype Cycle for AI in 2025 positions AI agents and AI-ready
data as the fastest advancing technologies.

Key Facts and Trends:
1. AI agents and autonomous systems are advancing rapidly, shifting
from generative AI capabilities to autonomous systems.
2. Agentic AI models, including GPT, Claude, Gemini, DeepSeek, Llama,
Grok, and more, are experiencing significant developments.
3. AI advancements in 2026 involve tools like Anthropic's agentic AI
tool and OpenAI's model routing refinement, demonstrating the growing
importance of AI agents.

These developments highlight the evolving nature of AI, moving from
generative AI to more autonomous systems and agentic AI models, with a
focus on enhancing technology and industry applications.
```

Step 4: Automated End-to-End Sequential Workflow (Stage 1 + Stage 2)

Using `RunnablePassthrough.assign()`, we chain Stage 1 and Stage 2 together into a single automated pipeline (`automated_research_workflow`). Passing a single `{"topic": "..."}` dictionary automatically triggers live web retrieval, injects the results into `{search_results}`, and generates the final summarized brief with zero manual intervention.

```python
from langchain_core.runnables import RunnablePassthrough

# Chain Stage 1 (Web Search) + Stage 2 (Summarization) into a single
```

```
automated pipeline
automated_research_workflow = (
    RunnablePassthrough.assign(
        search_results=lambda x: search_tool.invoke(x["topic"])
    )
    | summary_prompt
    | llm
    | StrOutputParser()
)

# Helper function to run any topic end-to-end with zero manual
intervention
def run_agent_workflow(topic: str):

print(f"============================================================")
    print(f" STARTING AUTOMATED WORKFLOW FOR TOPIC: '{topic}'")

print(f"============================================================\
n")

    final_brief = automated_research_workflow.invoke({"topic": topic})

    print(" FINAL AUTOMATED BRIEF:\n")
    print(final_brief)
    print(f"\
n============================================================")

# Test the complete automated pipeline
run_agent_workflow("Quantum Computing breakthroughs in healthcare")
```

```
============================================================
 STARTING AUTOMATED WORKFLOW FOR TOPIC: 'Quantum Computing
breakthroughs in healthcare'
============================================================

 FINAL AUTOMATED BRIEF:

Executive Summary: Quantum computing has the potential to
revolutionize healthcare by significantly enhancing computational
capabilities, leading to breakthroughs in drug design, clinical
trials, genome sequencing, and data security.

Key Facts:
1. Quantum computing has the potential to bring about supersonic drug
design, in silico clinical trials, and full-speed whole genome
sequencing and analytics, revolutionizing healthcare.
2. Quantum computing's impact on healthcare is still being explored,
with early breakthroughs in high-value problems, followed by gradual
expansion into adjacent applications as hardware matures and costs
decline.
```

```
3. Quantum computing can improve stakeholder collaboration in
healthcare solutions, particularly in the context of COVID-19, by
providing speed and accuracy, addressing the limitations of classical
computing.

These findings highlight the potential of quantum computing to
revolutionize healthcare, offering improved drug development, clinical
trial simulations, and genome sequencing, while enhancing data
security through quantum uncertainty.

============================================================
```

---

4. Problem-Solving Approach & Logic Implementation

A. Problem-Solving & Troubleshooting Approach

During the implementation and testing of this workflow on Linux Ubuntu (Python 3.12), four practical engineering challenges were encountered and systematically resolved:

1. **Dependency Isolation (`venv`):** Configured a dedicated Python virtual environment on Ubuntu so `langchain`, `langchain-openai`, `langchain-community`, and `ddgs` could be installed cleanly without conflicting with OS-level Python packages.
2. **API Quota & Model Discovery:** Standard OpenAI accounts without prepaid billing raise `HTTP 429 (insufficient_quota)` errors, and hardcoded model IDs can raise `HTTP 404 (model_not_found)` errors. We solved this by pointing `ChatOpenAI` to Groq's OpenAI-compatible base URL (`https://api.groq.com/openai/v1`) and dynamically querying `client.models.list()` to auto-select an available text generation model.
3. **Package Migration (`duckduckgo-search` → `ddgs`):** Updated `langchain-community` requires the renamed `ddgs` library for web search execution. Installing `ddgs` directly resolved the `ModuleNotFoundError`.
4. **Search Backend Timeout Mitigation:** Default search routing timed out when attempting to query throttled backends (`TimeoutException` on Mojeek). We engineered a multi-backend fallback loop (`["duckduckgo", "lite", "html", "bing", "brave"]`) with a 15-second timeout so retrieval succeeds reliably.

B. Logic Implementation Summary

- **Input Layer:** Accepts a single dictionary `{"topic": "<user_topic>"}`.
- **Automated Bridge (`RunnablePassthrough.assign`):** Preserves the original `topic` key while executing `search_tool.invoke(x["topic"])` via `RunnableLambda` and assigning the retrieved snippets to the `search_results` key.
- **Prompt & Inference Layer (`summary_prompt | llm | StrOutputParser()`):** Combines `{topic}` and `{search_results}` into a structured instruction prompt, invokes the LLM, and parses the output message into a clean string brief.

---

5. Observations from Task 6 (Short Note)

Observations:
Testing the end-to-end sequential workflow across multiple topics (*"Quantum Computing breakthroughs in healthcare"* and *"Autonomous AI Agents in software engineering"*) confirmed that chaining `RunnablePassthrough.assign()` with LCEL (`|`) completely eliminates manual intervention between web search and summarization. Unlike autonomous agents that dynamically loop and decide tool calls at runtime, this fixed two-stage pipeline executed deterministically with low latency and predictable token usage. Grounding the LLM prompt in real-time DuckDuckGo search snippets enabled accurate synthesis of recent developments beyond the model's static training cutoff while maintaining a consistent executive brief structure across different domains.